UNIVERSITY OF Southampton
School of Electronics and Computer Science

Comp1202: Introduction I

Welcome!

Welcome to Programming I

Thai **Son** Hoang
(T.S.Hoang@ecs.soton.ac.uk)

Heather Packer
(hp3@esc.soton.ac.uk)

Jian Shi
(s.snow@soton.ac.uk)

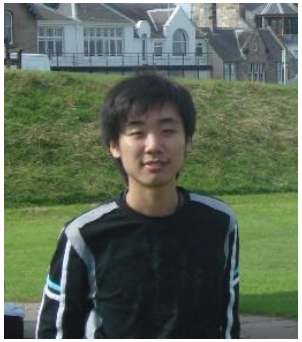
Purpose of this Module

"This module aims to introduce students to the **principles of programming** using an object oriented approach, and to provides them with the programming skills necessary to continue the study of computer science. **Java is used as the introductory language**."

- Writing and running programs
- Compilation, Interpretation and the Java Virtual Machine
- Variables, Objects, Primitives and Scope
- Methods
- Computational Thinking
- Constructors
- Loops and Arrays
- Collections and Iterators
- The Java Library
- Integrated Development Environments
- Testing and Debugging
- Software Design (What makes a good program)
- Super and Sub Classes (Inheritence)
- Polymorphism and Dynamic Binding
- Abstract Classes and Interfaces
- Designing Applications (Moving from problem to solution)

Main Course Structure

- Two Lectures Each Week
  - Monday 13:00 – 32/1015
  - Tuesday 16:00 – 32/1015



- Also a Lab each week (worth **20%**)
  - Wednesday 09:00 or 11:00
  - ECS Labs (Bld 59, Zepler, Level 2)
  - You have been allocated via timetabling
  - Details on next few slides

- Other Assessment
  - Coursework
    (set in week 4, due in week 10, worth **40%**)
  - Multiple Choice Exam (after X-mas, worth **40%**)

About Labs

- 10 labs every **Wednesday**
- **Starting October 2nd** (i.e., this week)
- Attend **your slot** allocated on your timetable, there are **no free spaces** in the other slot
- **Lab attendance counts towards your lab marks**
- You **MUST** attend **8 out of 10** labs to get full attendance marks
- You **MUST** record your attendance in the lab for it to count towards your marks

Code Functionality Test Harness

- Code Functionality is assessed using a **test harness**
- You will **receive an email** detailing any issues or whether you've passed
- **Make sure you follow the lab instructions carefully**, otherwise your code might not meet the spec
  - That includes spelling in outputs!
- You must **structure** your code **exactly** the same as specified in the lab otherwise the test harness won't work
- **Submit your code well before the deadline**, otherwise you might encounter an issue making you miss the deadline

Code Style Assessment

- Code Style is assessed in marking labs
- Marking labs happen on the 4th, 7th and 10th week
- We may not be able to mark everyone on those weeks, we will mark those left in the following week
- A random selection of the tasks from the lab will be marked, so be ready!

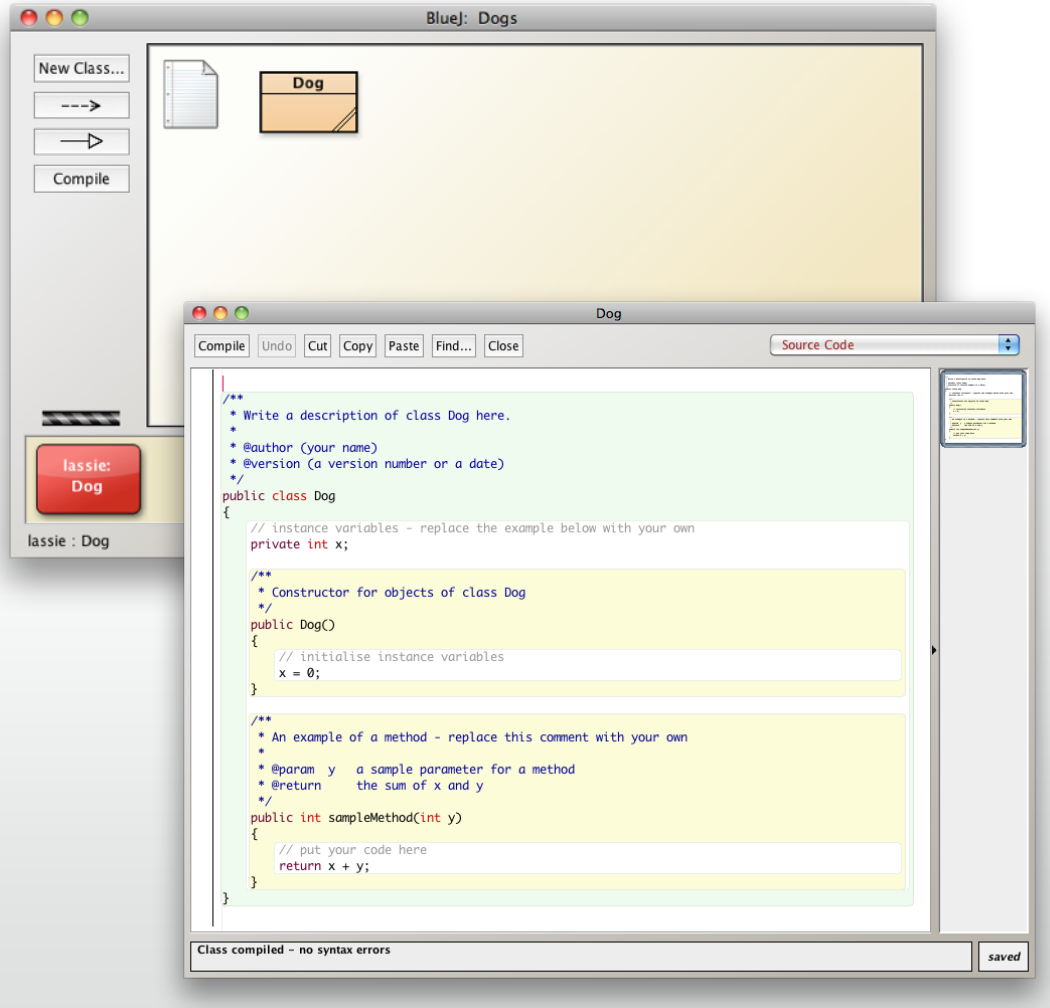
Additional Streams

- Space Cadets
  - For people who are more experienced programmers
  - Run by Son
  - Optional weekly challenge and discussion
    - Thursday 11:00 in 46/2005
    - Weeks 1-10 (i.e., starts this week!)

- Ground Controllers
  - For people who are new to programming
  - Run by Son and Jian
  - Optional weekly workshop
    - Tuesday 10:00 – 44/1061 (overflow 46/3101)
    - Weeks 2-10 (i.e., starts next week!)

BlueJ

- The main course text is *Objects First with BlueJ*
- Although we will start with using Java from the command line, we will soon introduce you to the BlueJ environment
- BlueJ is an environment designed for learning, but later in the course we will change over to use a proper professional environment



A Dirty Secret

- No matter how we teach you will mainly learn through practice!
- Programming is the single most important skill for a computer scientist or IT specialist
  - Systematic thinking and problem solving
  - Abstraction and data modeling
- Did we mention that you need to practice?

PRACTICE!

“I've often thought that sucking less every year is how humble programmers improve. You should be unhappy with code you wrote a year ago.”

- Jeff Atwood, http://www.codinghorror.com/
(Founder of stackoverfolow.com)

“I have no talent. What I do have is a lot of practice. And I am not talking about occasionally dabbling in Ruby on the weekends. I am talking about the kind of practice where I beat code that isn’t working into submission (though often times the code wins).”

- John Nunemaker, http://railstips.org/

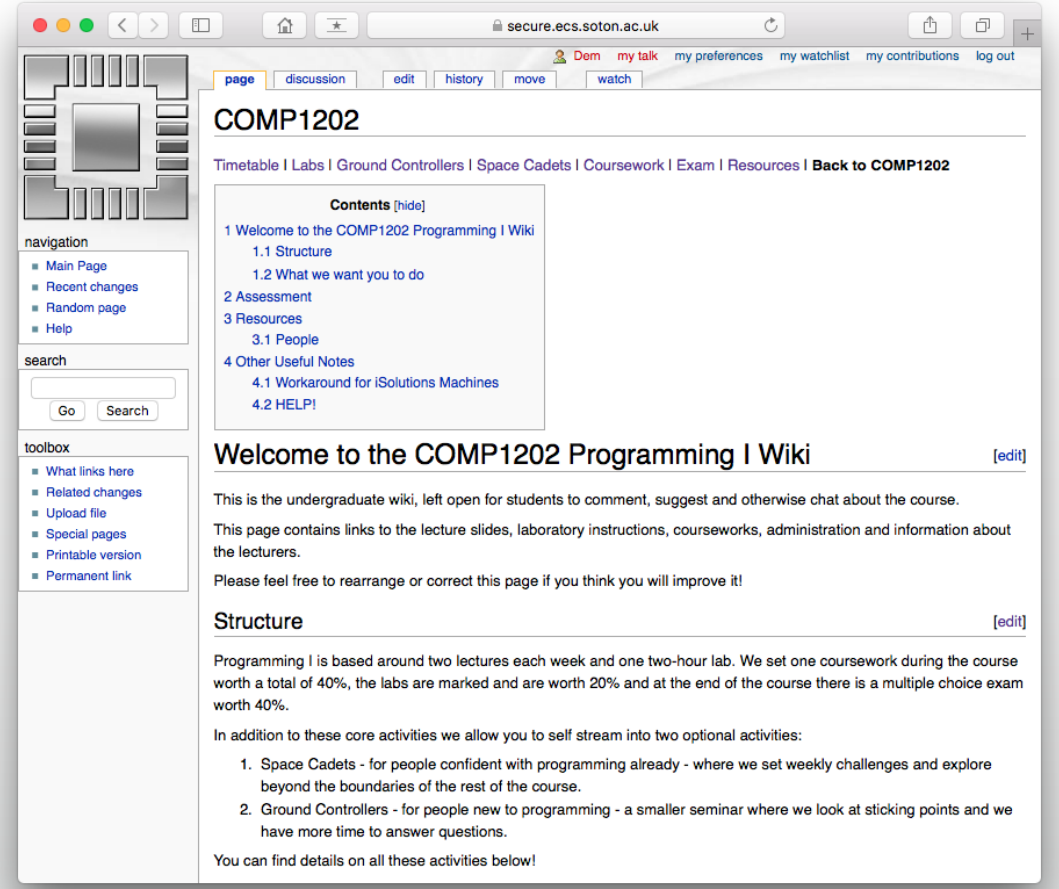
Your Lecturer: A Warning from History

Online Notes Wiki

https://secure.ecs.soton.ac.uk/module/comp1202/

leads to

https://secure.ecs.soton.ac.uk/student/wiki/w/COMP1202



Timetables

- Your timetable shows all of the sessions / labs allocated to your modules

- Modules do not always use all of those sessions!
  - For example, we have optional seminars

- So check the notes pages of each course for more info and adjust your timetable accordingly!

Reminder: Comp1202 Timetable

Week	Start Date	Topic	Section of Objects First Textbook	Mon 1pm (lecturer)	Tue 4pm (lecturer)	Tue 10am	Thu 11am	Lab - Wed 9am or Wed 11am
1	30/09/19	Introduction		Starting Out (sh, js)	Java (sh)	n/a	Space Cadets introduction	Hello World and Conditionals
2	07/10/19	Programming in Java	1.1, 1.2, 1.5, 2.15	Variables, Objects, Primitives and Scope (js)	Methods (js)	Ground Controllers Induction	Space Cadets	Classes, Conditionals and Control Statements
3	14/10/19	Building Better Programs/Objects	2.4	Computational Thinking (dem)	Constructors (js)	Ground Controllers	Space Cadets	Methods, Object Interaction and Testing
4	21/10/19	Loops and Arrays	4.9, 4.10	Loops and Arrays (js)	Collections and Iterators (js)	Ground Controllers	Space Cadets	Loops and ArrayLists
5	28/10/19	Environments and the Java Lib	4.12, 4.14.7	The Java Library (sh)	Changing IDE (sh)	Ground Controllers	Space Cadets	Arrays, HashMaps and APIs
6	04/11/19	Inheritance	8.2, 8.3, 8.4	Super and Sub-classes (sh)	Polymorphism (sh)	Ground Controllers	Space Cadets	Inheritance, Method Overwriting, Polymorphism and Exceptions
7	11/11/19	Building Better Classes	3.13	Testing and Debugging (sh) Testing and Debugging	Software Design (dem)	Ground Controllers	Space Cadets	Scope, Test Harnesses and Debugging
8	18/11/19	Object-Oriented Design	10.3, 10.4, 10.6, 10.7	Abstract Classes and Interfaces (sh)	Designing Applications (dem)	Ground Controllers	Space Cadets	Method Overloading, Interfaces and Class Hierarchy
9	25/11/19	Coding in Practice		Coding Dojo - part 1 (sh, js)	Coding Dojo - part 2 (sh, js)	Ground Controllers	Space Cadets	Handling Files
10	02/12/19	Other Languages	10.10	Languages Balloon Debate (sh)	NOT USED	n/a	Space Cadets	Marking Lab
11	09/12/19	Revision Lecture		NOT USED	Revision Lecture (sh, js)	n/a	n/a	

How to Get Help

- **Don't** email (get's lost in the avalanche)
- **Don't** drop by the office (hardly ever free)

- **Do** ask questions in the sessions
- **Do** come and speak to us at the end
- **Do** ask questions in the labs
- **Do** drop into help desk in Zepler Level 3
  - *10-12 and 2-5 Monday to Friday during term-time*
- **Do** ask each other!

Feedback

- You will get informal lab feedback every week
  - around 1 demonstrator per 10 students
- You will get formal lab feedback every 3 weeks
  - weeks 4, 7, 10
  - verbal feedback in the lab
  - written feedback (via email) shortly afterwards

- You will also get written feedback based on the coursework
  - Hand-in week 10
  - feedback in the new year, but before the exam

UNIVERSITY OF Southampton
School of Electronics and Computer Science

Comp1202: Introduction II

Starting Out

In this part

- What is Programming?
- Programming Paradigms

- A Taste of Things to Come
  - Classes and Objects
  - Variables
  - Logic
  - Data Structures
  - Skills

What is Programming?

- Wikipedia (2019)
  - “is the process of designing and building an executable computer program for accomplishing a specific computing task … The purpose of programming is to find a sequence of instructions that will automate the performance of a task for solving a given problem.”
- thefreedictionary.com (2019)
  - “creating a sequence of instructions to enable the computer to do something”
- dictionary.com (2019)
  - “the act or process of planning or writing a program”
- (OED) Oxford English Dictionary (2019)
  - “the writing or preparation of computer programs.”

What is Programming?

- Wikipedia (2019)
  - "is the process of designing and building an executable computer program for accomplishing a specific computing task … The purpose of programming is to find a sequence of instructions that will automate the performance of a task for solving a given problem."
- Thefreedictionary.com (2019)
  - "creating a sequence of instructions to enable the computer to do something"
- Dictionary.com (2019)
  - "the act or process of planning or writing a program"
- Oxford English Dictionary (2019)
  - "the writing or preparation of computer programs."

What is Programming?

- Wikipedia (2018)
  - "is the process of designing and building an executable computer program for accomplishing a specific computing task … The purpose of programming is to find a sequence of instructions that will automate the performance of a task for solving a given problem."
- Thefreedictionary.com (2018)
  - "creating a sequence of instructions to enable the computer to do something"
- Dictionary.com (2019)
  - "the act or process of planning or writing a program"
- Oxford English Dictionary (2019)
  - "The process of writing computer programs"

What is Programming?

- Wikipedia (2018)
  - "is the process of designing and building an executable computer program for accomplishing a specific computing task … The purpose of programming is to find a sequence of instructions that will automate the performance of a task for solving a given problem."
- Thefreedictionary.com (2018)
  - "creating a sequence of instructions to enable the computer to do something"
- Dictionary.com (2019)
  - "the act or process of planning or writing a program"
- Oxford English Dictionary (2019)
  - "the writing or preparation of computer programs."

Programming Flavours

- Procedural (e.g. C or Pascal)
  - Fixed list of instructions
  - Do this, Do that, if this then do that

- Declarative
  - More like declaring rules (or a grammar)
  - Behaviour emerges from the rules being applied
  - Examples
    - Functional Programming (e.g. Scheme or Haskell)
    - Logic Programming (e.g. Prolog)

Object Orientated

The main idea

- Everything is a Thing
- A Program is made up of Things interacting

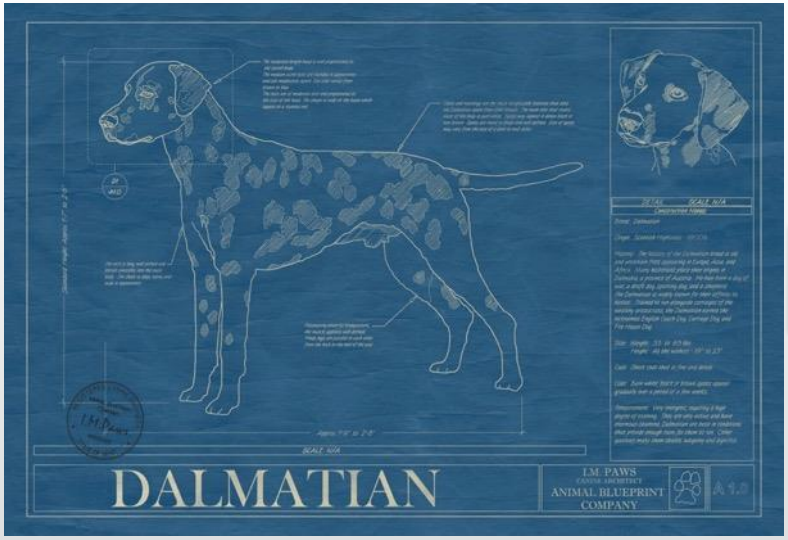
- Things have both properties and behaviours
- E.g. **Dogs**
  - **What are the properties of a dog?**
  - **What can a dog do?**

Classes and Objects

All the properties of Dog can be wrapped up or contained in a **class**

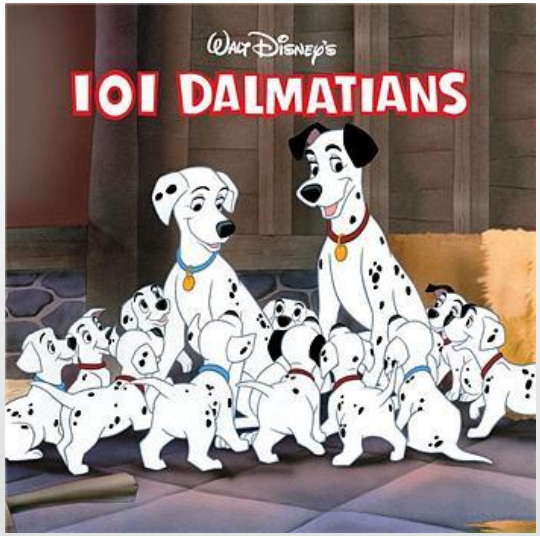
- A class is like a Blue Print
- We can build many unique dogs from the same Blue Print
- These are called **objects**
- They all have similar properties as defined by the class

1 Class



http://www.animalblueprintcompany.com/

101 Objects



Building Blocks

- Objects and Classes are specific to Object Orientated Programming

- But there are more common, more fundamental Programming Principles that we will be covering in the course

Variables

- Like algebra

  x = 4

- Not so much like algebra

  name = “Rover”

- Object properties are stored in variables

Methods

- Blocks of code that define a sequence of actions
- Object behavior is defined in methods
- Often this will use the object's properties

```
printDogsDetails()
{
  print name;
  print age;
}
```

Logic

True or False – the basis of all decisions

?	x = 3	Meaning x=4	x=11
x == 4	false	x is equal to 4 true	false
x > 4	false	x is greater than 4 false	true
x =< 4	true	x is smaller than or equal to 4 true	false
x != 4	true	x is not equal to 4 false	true
x > 2 && x < 8	true	x is greater than 2 and x is smaller than 8 true	false
x > 8 || x < 2	false	x is greater than 8 or x is smaller than 2 false	true

Control Flow

- If statements

```
if size<10
  dog says "Yip Yip"
else
  dog says "Ruff Ruff"
```

- Loops

```
while(number<10)
  dog says "Bark!"
  number= number +1
```

Side Note:

Human readable versions of programs are called **Pseudocode**.

They look like real programs but are not as precisely defined.

They are good for communicating ideas and showing structure.

Data Structures

- More complex data…

- Arrays:
  - Like Lists, Tables, Matrices
- HashMaps
  - Associate a key with a value
  - Like a Dictionary

Skills

- Computational Thinking
- Program Design
- Choosing Tools (IDEs)
- Testing/Debugging

Summary

- What is Programming?
- Programming Paradigms

- A Taste of Things to Come
  - Classes and Objects
  - Variables
  - Logic
  - Data Structures
  - Skills